

mdstats Interpreter Hot-Path Patching Manual

Completed fixes, patch standards, and deferred optimization roadmap

mdstats project

2026-07-22

Contents

Status and authority	2
Core engineering rule	2
Dense-loop test	3
Scientific invariants	3
Completed patch history	3
0.19.70a0 - Packed sparse-mesh reads	3
Failure	3
Patch	3
Evidence	4
Maintenance rule	4
0.19.71a0 - Stage-1 dense kernel cleanup	4
0.19.72a0 - Stage-2 shared hot paths	4
Standard patch workflow	5
Step 1 - Reproduce an end-to-end symptom	5
Step 2 - Localize the dominating stage	5
Step 3 - Classify the loop	6
Step 4 - Establish a scalar oracle	6
Step 5 - Design fixed-width data exchange	6
Step 6 - Chunk by the runtime budget	6
Step 7 - Benchmark alternatives	6
Step 8 - Integrate progress and resource accounting	7
Step 9 - Validate scientific equivalence	7
Step 10 - Add a regression guard	7
Approved implementation patterns	7
Fixed-width key encoding	7
Ragged numeric scheduling	8
Block/tile grouping	8
Sort-and-unique reconciliation	8
Existing compiled libraries first	8
Prohibited patterns	8
Deferred optimization roadmap	9
P0 - Multi-HDR field-level contour reuse	9
P0 - Sparse worker data transport	9
P1 - Tile-level parallel contour extraction	9
P1 - Remaining mesh component routines	9
P1 - HDR component labeling	10
P1 - Topology-statistics transition aggregation	10
P2 - I/O parsing	10
P2 - Framework topology projection	10
P2 - Primitive-ring enumeration	10

P2 - Natural tiling and periodic symmetry	11
Compiled-extension standard	11
Preferred boundary	11
Technology choice	11
Benchmark and release protocol	12
Representative benchmark set	12
Acceptance criteria	12
Release audit template	12
Static scan policy	13
Patch checklist	13
Current priority order	13
Maintenance map	14
Closing principle	14

Status and authority

Package baseline: mdstats 0.19.72a0

Document status: normative maintenance manual

Scope: Python-interpreter overhead in computationally demanding mdstats paths

This manual consolidates the completed interpreter-hot-path work from 0.19.70a0 through 0.19.72a0 and the remaining optimization plan. It is the operational guide for deciding whether a slow path should be vectorized, delegated to an existing compiled library, chunked, or moved behind a compiled mdstats extension.

The manual does **not** require every visible Python loop to be removed. It requires dense numerical work to leave the Python interpreter while preserving bounded Python orchestration and clear irregular graph algorithms.

The following project documents remain detailed implementation records:

- docs/specs/plotting/density_packed_mesh_read_ld13_spec.md
- docs/specs/performance/interpreter_hotpath_policy.md
- docs/specs/performance/interpreter_hotpath_policy.md
- audits/release/ld13_packed_mesh_read_audit.md
- audits/release/interpreter_overhead_01971_audit.md
- audits/release/interpreter_overhead_01972_audit.md
- audits/release/interpreter_hotpath_static_scan_01972.json
- audits/release/interpreter_hotpath_benchmarks.json
- audits/release/interpreter_hotpath_stage2_benchmarks.json

This manual supersedes those files only as the **maintenance workflow and roadmap**. Numerical details and release-specific evidence remain authoritative in their corresponding specifications and audits.

Core engineering rule

mdstats may coordinate scientific work in Python, but it must not execute dense numerical work one physical element at a time through the Python interpreter.

The preferred implementation order is:

1. NumPy broadcasting, indexing, sorting, reductions, and batched matrix multiplication;
2. SciPy FFT, sparse, graph, spatial, and signal kernels;
3. established compiled libraries already used by mdstats, including scikit-image marching cubes and fast-simplification;
4. bounded Python orchestration over fields, components, tiles, or memory-limited chunks;
5. a deterministic compiled mdstats extension for dominant irregular algorithms that cannot be expressed safely through existing compiled kernels.

`numpy.vectorize` and `numpy.frompyfunc` are not acceleration mechanisms. They retain Python function dispatch and are prohibited as substitutes for compiled numerical kernels.

Dense-loop test

A loop is presumptively a hot-path defect when its iteration count scales directly with one or more of:

- frames multiplied by atoms;
- grid nodes, voxels, or support nodes;
- source-stencil Gaussian contributions;
- neighbor candidate pairs;
- mesh vertices, faces, or edge occurrences;
- rows of fixed-width numerical records;
- repeated bit operations over packed numerical blocks.

The loop is normally acceptable when it operates over a small or deliberately bounded outer structure:

- chemical species or density fields;
- three Cartesian axes;
- a controlled number of compute/render tiles;
- memory-bounded chunks;
- connected components or topology states;
- user-facing Plotly traces;
- a fixed set of exact cases, such as the 27 active-set patterns used by the metric-box solver.

Scientific invariants

An optimization must preserve:

- periodic image and minimum-image conventions;
- exact integer shifts and canonical ordering;
- density estimator, Gaussian bandwidth, support tolerance, and normalization;
- topology, ring, tiling, and graph semantics;
- deterministic CSR, histogram, field, and mesh results;
- runtime memory, thread, and wall-time admission;
- progress-port behavior at stable stage boundaries;
- serialization compatibility unless a schema change is explicitly specified.

A faster result with changed scientific meaning is not an optimization; it is a new algorithm and requires a separate scientific specification.

Completed patch history

0.19.70a0 - Packed sparse-mesh reads

Failure

A tiled contour brick normally queries $33^3 = 35,937$ scalar nodes. The old packed-field reader handled each query independently and repeatedly decoded the same storage-block occupancy bitset. For a block containing P_b positive nodes, repeated queries paid approximately

$$O(Q_b P_b)$$

through Python-level bit operations.

Patch

Queries are now reduced and grouped by active storage block. Each touched block is decoded once, all requested local indices are searched together, and values are scattered back in original query order.

The dominant access cost becomes approximately

$$O(Q \log B + \sum_{b \in T} P_b),$$

where B is the active-block count and T is the set of touched active blocks.

Evidence

The reproducing benchmark improved from 25.7809 s to 0.02135 s for one fully occupied 33^3 tile, approximately 1,207x in the dominant operation. This explained the observed 1,264 s first-shell render almost exactly.

Maintenance rule

Packed scalar-field access must remain block-grouped. Do not add a convenience API that reintroduces one-node-at-a-time bitset decoding inside a large caller loop.

0.19.71a0 - Stage-1 dense kernel cleanup

The first package-wide pass replaced five dense interpreter paths:

Kernel	Historical problem	Current compiled path	Focused speedup
Packed bitset construction	Python bit/word loop	<code>numpy.bitwise_or.at</code>	10.29x
Packed bitset decoding	Python bit loop	<code>numpy.unpackbits</code>	18.32x
Bitset row popcount	Python/int object work	byte lookup and reductions	5.89x
Reverse sparse CSR	nested source-target loop	repeat, lexsort, bincount, cumsum	4.99x
Exact contour-vertex recovery	per-vertex Python geometry	batched NumPy reconstruction	7.97x
Frame-dependent coordinate transforms	frame loop over matrix products	broadcasted <code>numpy.matmul</code>	3.37x

The attempted `numpy.einsum` coordinate rewrite was rejected because it was slower for small 3 × 3 batched transforms. This establishes a project rule: **syntactic vectorization is not sufficient evidence; benchmark the candidate implementation.**

0.19.72a0 - Stage-2 shared hot paths

The second pass addressed larger architectural paths:

Area	Completed implementation	Focused speedup
Cell-list candidate expansion	bounded broadcast, occupied-bin lookup, encoded deduplication, chunked MIC evaluation	correctness-focused
Metric-aware stencil pruning	exact 27 active-set patterns evaluated across batches	6.17x
Bond-angle accumulation	ragged neighbor-pair templates, vector angle evaluation, compiled histogram reduction	28.19x
Sparse support atlas	fixed-width records, <code>bitwise_or.at</code> , sorted CSR, SciPy connected components	1.66x
Tiled mesh reconciliation	occurrence arrays and global sort/unique vertex/face welding	1.42x
Fragmented direct density	target-block ragged relation schedules and bounded vector chunks	correctness-focused

Only true canonical-cell polygon clipping remains in a per-triangle Python loop; it is irregular and normally applies to a small boundary subset.

Standard patch workflow

Every future hot-path patch should follow the same sequence.

Step 1 - Reproduce an end-to-end symptom

Record:

- input size and physical system;
- frame and atom counts;
- selected species and field count;
- resolved backend, grid, Gaussian width, and support tolerance;
- runtime memory, thread, and wall-time budgets;
- progress-port stage and elapsed time;
- final output size, mesh faces, and scientific diagnostics.

Do not optimize from static appearance alone. A loop can look suspicious and still be irrelevant to total wall time.

Step 2 - Localize the dominating stage

Use the package `ProgressPort` to identify the long stage. Add temporary nested timing only at coarse boundaries:

- input/normalization;
- connectivity/topology;
- field planning;
- field realization;
- scalar-brick gathering;
- contour extraction;
- mesh reconciliation;
- simplification;

- serialization.

Avoid timers inside every elementwise operation. They perturb the workload and create unusable logs.

Step 3 - Classify the loop

Classify the suspect path as one of:

1. **Dense fixed-width numeric:** replace with NumPy/SciPy/library kernel.
2. **Ragged numeric:** build offsets/CSR/segment arrays, then process values in compiled batches.
3. **Sparse fixed-width records:** encode records into integer arrays and use sort/search/unique/reduction.
4. **Bounded orchestration:** retain Python, but ensure the body delegates the actual arithmetic.
5. **Irregular graph/combinatorial:** profile and define a compiled-extension boundary; do not create combinatorially large dense tensors.

Step 4 - Establish a scalar oracle

Before replacing the path, preserve or write a small reference implementation that is:

- simple;
- deterministic;
- limited to small test inputs;
- independent enough to detect mistakes in the optimized kernel.

The oracle may remain in tests only. It must not become an accidental production fallback for large systems.

Step 5 - Design fixed-width data exchange

Prefer immutable arrays with explicit dtypes:

- int64 atom, node, face, or offset indices;
- int32 where bounds are proven and serialization benefits matter;
- uint64 packed occupancy words;
- float64 scientific coordinates and accumulated densities;
- CSR indptr and indices for ragged adjacency;
- fixed-column structured records only when plain 2-D arrays are insufficient.

Avoid large lists of tuples, dictionaries keyed by per-node objects, Python arbitrary-width integers for bulk bitsets, or per-record dataclasses in inner numerical paths.

Step 6 - Chunk by the runtime budget

Vectorization must not defeat resource management. Let

$$M_{\text{chunk}} = N_{\text{row}} b_{\text{row}} + M_{\text{workspace}}.$$

Choose N_{row} so that transient arrays remain below the active `RuntimeResourceBudget`, with reserve for retained fields and output geometry.

Chunk boundaries should be deterministic and reported in metadata when they materially affect execution planning. A patch must not allocate a full-cell dense tensor or all-pairs atom matrix merely to remove a visible Python loop.

Step 7 - Benchmark alternatives

Benchmark at least:

- current production path;
- proposed compiled/vectorized path;
- one representative small input;
- one representative large input;
- memory consumption or estimated transient bytes.

Reject a rewrite that is slower, less deterministic, or materially more memory-intensive without a compensating architectural reason.

Step 8 - Integrate progress and resource accounting

A new long stage must expose structured progress through `ProgressPort`. Resource planning must account for the optimized algorithm actually executed, not a nominal historical upper bound.

Report stable quantities such as:

- current/total fields, tiles, shells, or chunks;
- direct pair count;
- FFT padded nodes;
- candidate cells;
- raw and final faces;
- elapsed stage seconds.

Do not report every atom, grid node, pair, or triangle.

Step 9 - Validate scientific equivalence

Required tests depend on the subsystem but should include:

- scalar-oracle equivalence;
- random and adversarial inputs;
- empty and singleton cases;
- duplicate records;
- periodic boundary and corner cases;
- partial terminal blocks;
- deterministic ordering;
- serialization round trips;
- memory-limit and wall-time rejection;
- dense/sparse or old/new backend agreement where an oracle exists.

Step 10 - Add a regression guard

Use one or more of:

- AST checks preventing loops in registered dense kernels;
- benchmark threshold tests with generous hardware-independent ratios;
- call-count tests, such as one bitset decode per touched block;
- allocation-size assertions;
- structured progress/event assertions;
- planner metadata checks proving the actual executor cost is used.

The guard should prevent the exact architectural regression, not merely freeze one benchmark number.

Approved implementation patterns

Fixed-width key encoding

When keys have a small known number of integer columns, encode them into a 2-D array and use:

- `numpy.lexsort` for deterministic ordering;
- `numpy.unique(axis=0, return_inverse=True, return_counts=True)` for welding/deduplication;
- `numpy.searchsorted` for membership and mapping;
- `numpy.bincount` and `numpy.cumsum` for CSR construction;
- `numpy.minimum.at`, `numpy.maximum.at`, `numpy.add.at`, or `numpy.bitwise_or.at` for grouped reductions.

This pattern replaced Python dictionaries in support-atlas and mesh-reconciliation paths.

Ragged numeric scheduling

Represent variable-length groups by:

- indptr offsets;
- flat value/index arrays;
- group IDs generated by repeat or prefix expansion;
- bounded chunks over the flat arrays.

This pattern is preferred for bond angles, neighbor candidates, and direct sparse source-stencil schedules.

Block/tile grouping

Group many queries by storage block or render tile, decode/gather once, and process the local brick through compiled operations. Never repeat expensive block metadata decoding per node.

Sort-and-unique reconciliation

For globally repeated mesh vertices, faces, edges, or sparse records:

1. concatenate occurrence arrays;
2. sort or unique fixed-width keys;
3. map occurrences to canonical rows;
4. verify repeated-coordinate agreement;
5. reject degeneracy in arrays;
6. restore deterministic ordering.

Use Python dictionaries only for small irregular exceptions, not the dominant occurrence stream.

Existing compiled libraries first

Before creating new native code, check whether the operation is already provided by:

- NumPy;
- SciPy sparse/graph/spatial/FFT/signal;
- scikit-image;
- fast-simplification;
- NetworkX only for small orchestration or correctness, not large dense graph kernels.

Prohibited patterns

The following require explicit justification and benchmark evidence:

- Python loops over every atom pair, grid node, stencil contribution, mesh vertex, or face;
- repeated conversion between arrays and Python lists/tuples in inner stages;
- Python dictionaries or sets with one entry per dense numerical record;
- per-node bitset decoding;
- object-dtype NumPy arrays;
- `numpy.vectorize` or `frompyfunc` as performance fixes;
- unbounded task submission to process or thread pools;
- oversubscription between Python workers and BLAS/OpenMP pools;
- full dense allocation to avoid a sparse orchestration problem;
- a new cache whose build cost or retained memory is excluded from resource planning;
- changing Gaussian width, grid resolution, cutoff, or topology semantics merely to meet a performance target.

Deferred optimization roadmap

The roadmap is divided into work that is likely to benefit the density-rendering pipeline immediately and work that requires profiling before implementation.

P0 - Multi-HDR field-level contour reuse

Status: deferred; high value for interactive density rendering.

Current rendering commonly requests three HDR levels per field. Each shell can repeat field serialization, tile-brick gathering, and worker startup.

Target design:

1. create one isolated worker per field rather than per shell;
2. map or deserialize the packed field once;
3. gather each tile brick once;
4. evaluate all requested HDR levels from that brick;
5. emit per-level occurrence streams;
6. reconcile and simplify each shell independently;
7. preserve deterministic shell ordering and per-shell resource diagnostics.

Expected benefit: remove repeated field transfer and repeated tile reads across three shells. The patch must measure whether marching cubes or scalar gathering dominates after 0.19.70a0.

Validation requirements:

- identical mesh topology and values to independent-shell extraction;
- field-worker timeout and memory accounting;
- progress events for field and level completion;
- correct failure isolation when one level exceeds its face budget.

P0 - Sparse worker data transport

Status: deferred; profile after multi-HDR reuse.

Large fields should not be serialized separately for every shell. Candidate transports:

- one field worker retaining the field for all levels;
- read-only memory mapping of packed arrays;
- shared-memory arrays with explicit lifecycle ownership.

cloudpickle should remain orchestration support, not the primary transport for repeated multi-hundred-megabyte numerical payloads.

P1 - Tile-level parallel contour extraction

Status: deferred; only after field-level reuse.

Independent contour tiles may be processed concurrently when:

- the runtime thread budget prevents native oversubscription;
- per-worker scalar-brick and mesh workspace fit the memory budget;
- deterministic global reconciliation remains independent of completion order;
- process startup and transport do not exceed the saved compute time.

A bounded work queue is required. Do not submit every tile at once.

P1 - Remaining mesh component routines

Static scanning continues to flag irregular mesh paths, notably:

- `plotting.density_sparse_mesh._mesh_nonwinding_components`;

- `plotting.density_mesh_simplify._canonicalize_lifted_components`;
- canonical-cell clipping and winding/component validation.

These are candidates, not confirmed defects. Profile using scenes with many periodic components and record time separately from marching cubes and global reconciliation.

Possible patch direction:

- encode component memberships and lifted/canonical cell indices as arrays;
- use CSR connected components where semantics match;
- retain exact clipping predicates in Python or compiled native code;
- avoid dense component-by-cell matrices.

P1 - HDR component labeling

Static scanning flags `plotting.density_hdr._component_labels`. If profiling shows it dominates for fragmented support, replace explicit graph traversal with a SciPy sparse connected-component kernel or a compiled union-find over fixed-width adjacency.

Required invariant: identical connectivity under periodic wrapping and deterministic component labels.

P1 - Topology-statistics transition aggregation

`analysis.topology_statistics.atomic._compute_transition_aggregates` contains sequence-oriented Python work. Profile large state catalogs. If material, replace per-position accumulation with segmented reductions over transition IDs and frame ranges.

This should not be prioritized for ordinary trajectories unless profiling shows it contributes meaningful wall time.

P2 - I/O parsing

Trajectory parsers necessarily coordinate line-oriented text, but dense numeric conversion inside parsed blocks should use compiled conversion or bulk array construction. Profile before changing:

- LAMMPS dump frame scanning;
- concatenated VASP trajectory parsing;
- repeated symbol/type conversion.

Possible future approaches include memory-mapped byte scanning, compiled tokenizers, or optional binary/HDF5 paths. Parser changes require especially strong malformed-input and diagnostic tests.

P2 - Framework topology projection

`analysis.framework_topology.build_framework_topology` is an irregular path traversal over vertex/linker roles and periodic shifts. It should remain readable Python until representative large framework systems show projection time to be material.

If compiled:

- input should be immutable CSR adjacency plus integer periodic shifts and role arrays;
- path rules should be encoded as compact deterministic tables;
- output should be fixed-width edge/path arrays before conversion to public dataclasses;
- exact canonical ordering and digest stability are mandatory.

P2 - Primitive-ring enumeration

The static scan identifies even/odd candidate generation, shortest-path indexing, and external-shortcut witnesses as complex loops. These are combinatorial rather than dense numerical kernels.

Compiled-extension candidate boundary:

- CSR/lifted-graph input;

- bounded-depth BFS or shortest/second-shortest path kernels;
- integer periodic-shift state;
- candidate-cycle arrays;
- Python retains high-level validation, canonicalization, provenance, and API construction.

Do not replace ring search with dense adjacency powers or tensors whose memory grows exponentially with the length cutoff.

P2 - Natural tiling and periodic symmetry

Candidate modules include:

- `analysis.lta_natural_tiling` face certificates and tile shells;
- `analysis.natural_tiling` action-composition validation;
- `analysis.net_symmetry_discovery` generator and operation search;
- `analysis.periodic_net_view` component analysis.

These require representative profiling and likely a shared deterministic native graph kernel rather than separate local vectorizations.

Compiled-extension standard

A native extension should be introduced only when all conditions hold:

1. representative profiling shows the irregular Python kernel materially affects end-to-end runtime;
2. NumPy/SciPy/library kernels cannot express the operation without excessive memory or semantic loss;
3. the input/output boundary can be expressed with immutable arrays, CSR, fixed-width records, and scalar options;
4. a small Python oracle exists;
5. deterministic ordering and exact periodic integers can be preserved;
6. wheels can be built for supported platforms or a correct Python fallback remains available.

Preferred boundary

The Python layer should:

- validate public inputs;
- normalize arrays and dtypes;
- enforce resource limits;
- emit progress events;
- call one coarse native kernel;
- validate output invariants;
- construct public immutable result objects.

The native layer should:

- avoid Python callbacks inside the hot loop;
- release the GIL for long computation;
- accept explicit thread and memory limits;
- use deterministic schedules and reductions;
- return error codes or structured exceptions with enough context for Python diagnostics.

Technology choice

Choose based on the kernel, not preference:

- **Cython**: lowest-friction migration of existing typed loops and NumPy buffers;
- **C++/pybind11**: suitable for graph/geometry kernels and established C++ libraries;
- **Rust/PyO3**: suitable when memory safety and deterministic ownership justify the build complexity;

- **Numba:** optional experimentation only if packaging, warm-up cost, determinism, and supported environments are acceptable.

No technology is mandated by this manual. The ABI and test contract matter more than the language.

Benchmark and release protocol

Representative benchmark set

Maintain at least four classes:

1. small deterministic synthetic oracle case;
2. medium realistic case that runs quickly in CI;
3. large local workstation stress case;
4. full scientific benchmark, such as the complete LTA trajectory, run for release evidence when dependencies and resources are available.

Record:

- hardware and process allocation;
- package and dependency versions;
- input dimensions;
- resolved resource budget;
- backend and algorithm choices;
- wall time by progress stage;
- peak memory where available;
- scientific equivalence metrics;
- output complexity.

Acceptance criteria

A patch is accepted when:

- scientific and deterministic tests pass;
- no resource guard is weakened to make the benchmark pass;
- the optimized path is faster on the intended representative workload;
- memory remains within the runtime-derived budget;
- small-input overhead is acceptable or a thresholded fallback is retained;
- failure messages identify the actual limiting resource or stage;
- documentation and progress stages are updated.

A microbenchmark speedup alone is insufficient if end-to-end runtime does not improve.

Release audit template

Every hot-path release audit should contain:

Scope

Observed end-to-end symptom

Root cause

Historical complexity

New algorithm and complexity

Scientific invariants

Resource-accounting changes

Progress-reporting changes

Correctness tests

Performance benchmarks

Known limitations

Static scan policy

`tools/performance/scan_interpreter_hotpaths.py` is a discovery tool, not an automatic defect classifier. Its output should be reviewed in this order:

1. functions with loops scaling over dense physical records;
2. deeply nested loops in plotting/density/neighbor modules;
3. repeated object construction in numerical paths;
4. graph algorithms with unexpectedly large representative workloads;
5. parsers and presentation code last.

The raw finding count is not a quality metric. A lower count achieved by hiding loops in helper functions is meaningless. The goal is to remove interpreter work from measured hot paths while retaining readable architecture.

Patch checklist

Before implementation:

- ☐ Reproduce the slow stage with progress output.
- ☐ Capture input size, budgets, backend, and scientific parameters.
- ☐ Profile or time coarse sub-stages.
- ☐ Classify the loop: dense, ragged, sparse-record, orchestration, or irregular graph.
- ☐ Preserve a scalar/reference oracle.
- ☐ Estimate transient memory for the proposed vectorized path.

During implementation:

- ☐ Use fixed-width arrays/CSR for the numerical exchange boundary.
- ☐ Keep Python loops bounded over fields, tiles, components, or chunks.
- ☐ Apply runtime memory/thread/wall-time limits to the actual algorithm.
- ☐ Avoid object dtype and per-record Python containers.
- ☐ Add structured progress at stable stage boundaries.
- ☐ Preserve deterministic ordering.

Before release:

- ☐ Run oracle-equivalence and adversarial tests.
- ☐ Run periodic boundary/corner cases where applicable.
- ☐ Run resource rejection tests.
- ☐ Benchmark old and new paths on representative sizes.
- ☐ Add a regression guard for the architectural failure.
- ☐ Update this manual's completed-patch table and roadmap.
- ☐ Update the detailed subsystem specification.
- ☐ Record validation limitations honestly.

Current priority order

The recommended sequence after 0.19.72a0 is:

1. rerun the complete LTA interactive-density scene and capture stage timings;
2. implement multi-HDR field-level contour reuse if shell extraction remains significant;
3. eliminate repeated field serialization through one-worker-per-field or read-only mapped transport;
4. consider bounded tile parallelism only after transport and reuse are fixed;

5. profile remaining HDR/component mesh routines;
6. profile topology statistics and parsers only on workloads where they matter;
7. profile primitive-ring, tiling, topology-projection, and symmetry searches on representative large periodic graphs;
8. introduce one shared deterministic compiled graph extension only when profiling justifies it.

The next patch should be selected from measured wall time, not from static-scan rank alone.

Maintenance map

- **Interpreter policy:** docs/specs/performance/interpreter_hotpath_policy.md
- **Stage-2 implementation:** docs/specs/performance/interpreter_hotpath_policy.md
- **Packed sparse reads:** docs/specs/plotting/density_packed_mesh_read_ld13_spec.md
- **Static scan:** tools/performance/scan_interpreter_hotpaths.py
- **Progress interface:** docs/specs/progress_spec.md
- **Runtime resources:** docs/specs/plotting/density_runtime_resource_policy_ld10_spec.md
- **Density architecture:** docs/arch_manuals/mdstats_dynamical_framework_density_architecture_standard.md
- **This maintenance manual:** docs/arch_manuals/mdstats_interpreter_hotpath_patching_manual.md

Closing principle

The objective is not to make mdstats “look vectorized.” The objective is to keep scientific orchestration clear while ensuring that work proportional to large physical data sets executes in compiled, resource-bounded kernels. Every patch must be selected by representative profiling, preserve scientific invariants, and leave a regression guard that prevents the same interpreter-bound architecture from returning.